On the other hand, if you're generating C# clients for calling your API or exposing your API for public consumption, the more metadata you add, the better! It's also worth noting that you can add OpenAPI descriptions for all the endpoints in your application, not only your minimal API endpoints. When you create web API controllers in chapter 20, you can include them, too.

## 11.6 Knowing the limitations of OpenAPI

In this chapter I've described the benefits of OpenAPI, both for simple testing with Swagger UI and for code generation. But like most things in software, it's not all sweetness and light. OpenAPI and Swagger have limitations that you may run into, particularly as your APIs increase in complexity. In this section I describe some of the challenges to watch out for.

### 11.6.1 Not all APIs can be described by OpenAPI

The OpenAPI specification is meant to describe your API so that any client knows how to call it. Unfortunately, OpenAPI can't describe all APIs, which isn't an accident. The OpenAPI specification says "Not all services can be described by OpenAPI—this specification is not intended to cover every possible style of REST APIs." So, the important question is which APIs *can't* it describe?

One classic example is an API that follows the REST design known as Hypertext As the Engine of Application State (HATEOAS). In this design, each request to an API endpoint includes a list of links describing the actions you can take and the paths to use for each action, enabling clients to discover which actions are available for a given resource. The server can add or remove links dynamically, depending on the state of the resource and which user is making the request.

**TIP** Martin Fowler has a great description of the REST maturity models, in which HATEOAS is the highest level of maturity, at http://mng.bz/0K1N.